```cpp
#ifndef MATRIX_HPP
#define MATRIX_HPP
#pragma once
#include <iostream>
#include <vector>
#include <random>
#include <stdexcept>
#include <iomanip>
#include <cmath>
#include <functional>
#include <fstream>

class Matriz {
private:
    size_t filas_;
    size_t columnas_;
    std::vector<double> datos_; // Bloque contiguo 1D en memoria

public:
    // 1. Constructores
    Matriz() : filas_(0), columnas_(0) {}

    Matriz(size_t filas, size_t columnas, double init_val = 0.0)
        : filas_(filas), columnas_(columnas), datos_(filas * columnas, init_val) {}

    // 2. Getters de dimensiones
    size_t filas() const {
         return filas_;
    }
    size_t columnas() const {
         return columnas_;
    }

    // 3. Acceso 2D mapeado a 1D: M(i, j)
    double& operator()(size_t r, size_t c) {
        return datos_[r * columnas_ + c];
    }

    const double& operator()(size_t r, size_t c) const {
        return datos_[r * columnas_ + c];
    }

    // 4. Inicialización aleatoria (Semilla Mersenne Twister para pesos de IA)
    void aleatorizar(double min_val = -1.0, double max_val = 1.0) {
        static std::mt19937 gen(std::random_device{}());
        std::uniform_real_distribution<double> dist(min_val, max_val);
        for (double& val : datos_) {
            val = dist(gen);
        }
    }

    // 5. Suma y Resta elemento a elemento
    Matriz operator+(const Matriz& A) const {
        if (filas_ != A.filas_ || columnas_ != A.columnas_) {
            throw std::invalid_argument("Dimensiones incompatibles en suma (+).");
        }
        Matriz resultado(filas_, columnas_);
        for (size_t i = 0; i < datos_.size(); ++i) {
            resultado.datos_[i] = datos_[i] + A.datos_[i];
        }
        return resultado;
    }

    Matriz operator-(const Matriz& A) const {
        if (filas_ != A.filas_ || columnas_ != A.columnas_) {
            throw std::invalid_argument("Dimensiones incompatibles en resta (-).");
```

```
    }
    Matriz resultado(filas_, columnas_);
    for (size_t i = 0; i < datos_.size(); ++i) {
        resultado.datos_[i] = datos_[i] - A.datos_[i];
    }
    return resultado;
}
Matriz operator*(double A) const{
    Matriz resultado(filas_,columnas_);
    for(size_t i = 0; i < filas_*columnas_; i++){
        resultado.datos_[i] = datos_[i] * A;
    }
    return resultado;
}

// 6. Multiplicación Matricial (Producto Punto) optimizada para Caché (Orden i-k-j)
Matriz matmul(const Matriz& A) const {
    if (columnas_ != A.filas_) {
        throw std::invalid_argument("Dimensiones incompatibles en matmul.");
    }
    Matriz resultado(filas_, A.columnas_, 0.0);

    for (size_t i = 0; i < filas_; ++i) {
        for (size_t k = 0; k < columnas_; ++k) {
            double a_ik = (*this)(i, k); // Se mantiene en registro del CPU
            for (size_t j = 0; j < A.columnas_; ++j) {
                resultado(i, j) += a_ik * A(k, j); // Acceso contiguo
            }
        }
    }
    return resultado;
}

// 7. Transpuesta (Vital para Regresión Lineal y Backpropagation)
Matriz transpuesta() const {
    Matriz resultado(columnas_, filas_);
    for (size_t i = 0; i < filas_; ++i) {
        for (size_t j = 0; j < columnas_; ++j) {
            resultado(j, i) = (*this)(i, j);
        }
    }
    return resultado;
}

Matriz hadamard(const Matriz& A) const{
    Matriz resultado(filas_,columnas_,0.0);
    if(columnas_ == A.columnas_ && filas_ == A.filas_){
        for(size_t i = 0; i < filas_*columnas_; i++){
                resultado.datos_[i] = datos_[i] * A.datos_[i];
        }
    }else
        throw std::invalid_argument("Dimensiones incompatibles");
    return resultado;

}

Matriz aplicar(std::function<double(double)> func) const{
    Matriz resultado(filas_,columnas_);
    for(size_t i = 0; i < filas_*columnas_; i++){
        resultado.datos_[i] = func(datos_[i]);
    }
    return resultado;
}

Matriz softmax() const {
    Matriz resultado(filas_, columnas_);
```

```
        // 1. Encontrar el valor máximo para estabilidad numérica
        double max_val = datos_[0];
        for (size_t i = 1; i < datos_.size(); ++i) {
            if (datos_[i] > max_val) max_val = datos_[i];
        }

        // 2. Calcular los exponenciales y sumarlos
        double suma_exp = 0.0;
        for (size_t i = 0; i < datos_.size(); ++i) {
            resultado.datos_[i] = std::exp(datos_[i] - max_val);
            suma_exp += resultado.datos_[i];
        }

        // 3. Dividir cada exponencial entre la suma (Normalización)
        for (size_t i = 0; i < datos_.size(); ++i) {
            resultado.datos_[i] /= suma_exp;
        }

        return resultado;
}
    // 8. Utilidad para imprimir en consola
    void imprimir(const std::string& name = "Matriz") const {
        std::cout << name << " (" << filas_ << "x" << columnas_ << "):\n";
        for (size_t i = 0; i < filas_; ++i) {
            std::cout << "  [ ";
            for (size_t j = 0; j < columnas_; ++j) {
                std::cout << std::setw(8) << std::fixed << std::setprecision(4) << (*this
)(i, j) << " ";
            }
            std::cout << "]\n";
        }
        std::cout << "\n";
    }

    void guardarArchivo(std::ofstream& out)const{
        out << filas_ << " " << columnas_ << "\n";
        for(size_t i = 0; i < filas_*columnas_; i++){
            out << datos_[i] << " ";
        }
        out << "\n";

    }
    void cargarArchivo(std::ifstream& in) {
        size_t f, c;
        in >> f >> c;
        // (Aquí deberías redimensionar tu matriz si es necesario: inicializar(f, c))
        for (size_t i = 0; i < filas_*columnas_; ++i) {
            in >> datos_[i];
        }
    }
    // Agregar dentro de class Matriz en matrix.hpp
    void guardarBinario(std::ofstream& out) const {
        out.write(reinterpret_cast<const char*>(&filas_), sizeof(filas_));
        out.write(reinterpret_cast<const char*>(&columnas_), sizeof(columnas_));
        out.write(reinterpret_cast<const char*>(datos_.data()), datos_.size() * sizeof(
double));
    }

    void cargarBinario(std::ifstream& in) {
        in.read(reinterpret_cast<char*>(&filas_), sizeof(filas_));
        in.read(reinterpret_cast<char*>(&columnas_), sizeof(columnas_));
        datos_.resize(filas_ * columnas_);
        in.read(reinterpret_cast<char*>(datos_.data()), datos_.size() * sizeof(double));
}
};
```

```
#endif // MATRIX_HPP
```